Laboratorio 7 — Hoja de trabajo

Excepciones, front controller y plantillas

Nombre: Ronald Salvatierra Flores **Fecha:** 14 / 10 / 2026

Comandos: `composer dump-autoload` · `php verificar.php` (5 de 15 al empezar, 15 de 15 al terminar) · servidor con router: `php -S 0.0.0.0:8000 -t public public/index.php` (en Docker). Las rutas de archivo de los mensajes son las del contenedor (/app).

Parte A — Una sola puerta de entrada

1. Abre /, /producto/MIE-1K, /cotizacion y /api/articulos. ¿Qué archivo PHP atiende todas esas URL?

public/index.php, el front controller. Las cuatro responden 200 y todas pasan por ese único archivo, porque el servidor arranca con el router como último argumento (php -S … -t public public/index.php). Desde ahí el Router decide qué método del controlador ejecutar.

2. Abre /css/estilo.css. ¿Qué responde antes del TODO 4? ¿Por qué? Completa el TODO 4.

```
Antes del TODO 4:  HTTP/1.1 404 Not Found
Después:           HTTP/1.1 200 OK   (Content-Type: text/css)
```

Como index.php es el router, todas las peticiones entran a él, también la del archivo CSS. El Router no tiene una ruta /css/estilo.css y lanza NoEncontrado (404), por eso la página se veía sin estilos. Solución: si PHP_SAPI es cli-server y la ruta es un archivo real dentro de public/, se devuelve false para que el servidor lo entregue tal cual.

```
if (PHP_SAPI === 'cli-server') {
    $archivo = realpath(__DIR__ . rawurldecode((string) parse_url($_SERVER['REQUEST_URI'],
PHP_URL_PATH)));
    if ($archivo !== false && str_starts_with($archivo, realpath(__DIR__) . DIRECTORY_SEPARATOR)
&& is_file($archivo)) {
        return false;
    }
}
```

Se usa realpath() y str_starts_with() para que nadie pueda salirse de public/ con "../".

3. En producción con Nginx (Tecnología Web I), ¿qué directiva cumple la misma función?

```
location / { try_files $uri /index.php$is_args$args; }
```

try_files busca primero un archivo real con esa ruta ($uri: CSS, imágenes) y, si no existe, envía la petición a /index.php conservando la cadena de consulta. Es lo mismo que hace el TODO 4 con php -S.

4. ¿Qué expresión regular genera el Router para /producto/{codigo}? ¿Cómo llega MIE-1K al controlador?

```
#^/producto/(?P<codigo>[^/]+)$#
```

Router::agregar() reemplaza cada {nombre} por un grupo con nombre (?P<nombre>[^/]+): cualquier texto sin "/". Al despachar, preg_match deja en $coincide tanto claves numéricas como la clave con nombre; array_filter(..., 'is_string', ARRAY_FILTER_USE_KEY) conserva solo las claves con nombre (codigo => MIE-1K) y se llama al controlador con ($r['accion'])(...$parametros), es decir, con argumento con nombre: producto(codigo: 'MIE-1K').

Parte B — Excepciones con significado (TODO 1, 2 y 5)

1. Dibuja la jerarquía: RuntimeException → ErrorAplicacion → …

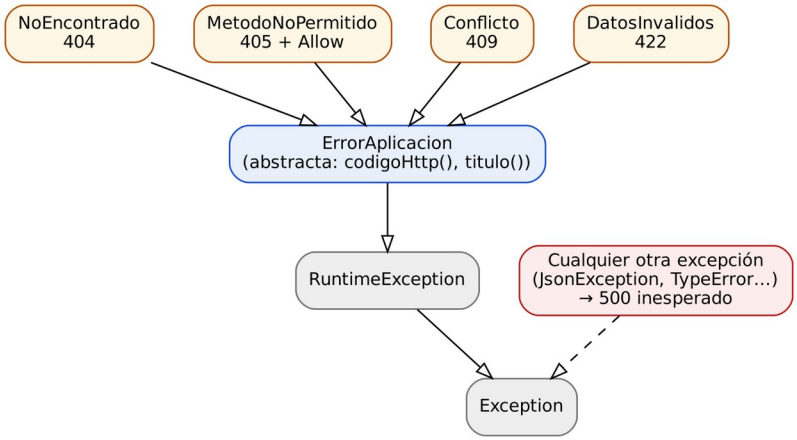


Figura 1. Jerarquía de excepciones del laboratorio.

2. TODO 1: ¿qué código HTTP corresponde a Conflicto? Justifícalo.

```
public function codigoHttp(): int
{
    return 409;
}
```

409 Conflict. La petición está bien formada y es válida (no es un 400 ni un 422), pero choca con el estado actual del recurso: se pidieron 5 unidades de CER-1K y solo hay 4. Antes respondía 500, que significa "falló el servidor", y eso no es cierto: el servidor funciona y la petición simplemente no se puede cumplir.

3. Antes del TODO 2, envía curl -i -X POST http://127.0.0.1:8000/. ¿Qué código devuelve? ¿Cuál debería devolver y con qué cabecera?

```
Antes:   HTTP/1.1 404 Not Found
Después: HTTP/1.1 405 Method Not Allowed
         Allow: GET
```

Antes devolvía 404. Debería devolver 405 con la cabecera Allow: GET, porque la ruta / sí existe pero solo acepta GET. Con el TODO 2 el Router anota los métodos de las rutas que coinciden ($permitidos[] = $r['metodo']), lanza MetodoNoPermitido y index.php agrega la cabecera Allow.

4. Antes del TODO 5, cotiza 5 unidades de CER-1K (hay 4). Copia lo que muestra la página. ¿Qué dos problemas ves?

```
HTTP/1.1 500 Internal Server Error
500
DomainException
Stock insuficiente de CER-1K: hay 4, se pidieron 5. en /app/src/Catalogo.php:83
Volver al catálogo
```

Problema 1: responde 500 (error del servidor) para algo que es un error previsto de la petición; debería ser 409. Problema 2: muestra información interna al usuario: la clase de la excepción (DomainException) y la ruta del archivo con el número de línea. Eso ayuda a un atacante y no le sirve a quien cotiza.

5. Después del TODO 5: ¿para qué sirve pasar $e como tercer argumento ($previous)?

```
try {
    $this->catalogo->verificarStock($pedido);
} catch (DomainException $e) {
    throw new Conflicto($e->getMessage(), 0, $e);
}
```

Sirve para encadenar las excepciones: Conflicto es lo que entiende la capa HTTP (409), pero la DomainException original, con su archivo, línea y traza, queda guardada y se recupera con getPrevious(). Así se traduce el error al lenguaje HTTP sin perder la causa real para depurar. Ahora la página responde:

```
HTTP/1.1 409 Conflict
No se puede completar
Stock insuficiente de CER-1K: hay 4, se pidieron 5.
```

Parte C — Plantillas con escape automático (TODO 3)

1. Busca <script>alert(1)</script> en el catálogo antes del TODO 3. ¿Qué pasa?

```
17:  <input id="q" name="q" type="search" value="<script>alert(1)</script>">
20:<p>Resultados para «<script>alert(1)</script>»: 0</p>
```

El texto se inserta tal cual en el HTML (en el código fuente aparece un <script> real). El navegador lo ejecutaría: es una vulnerabilidad XSS.

2. Completa el TODO 3. ¿Por qué las plantillas ya no necesitan e()?

```
is_string($valor) => htmlspecialchars($valor, ENT_QUOTES, 'UTF-8'),

17:  <input id="q" name="q" type="search" value="&lt;script&gt;alert(1)&lt;/script&gt;">
```

Plantilla::escapar() recorre recursivamente todos los datos antes de pasarlos a la plantilla y escapa cada texto. El escape es automático y seguro por defecto: ya no depende de que el programador se acuerde de llamar e() en cada variable, donde un solo olvido sería un agujero de seguridad.

3. ¿Qué es Crudo y cuándo es correcto usarlo? ¿Cuándo sería peligroso?

Crudo es un envoltorio que marca un texto como "HTML confiable": escapar() lo deja pasar sin tocarlo. Es correcto con HTML que genera la propia aplicación, por ejemplo el contenido ya renderizado de otra plantilla que se inserta en base.php ($contenido). Es peligroso con datos que escribe el usuario (la búsqueda, un nombre, un comentario), porque reabre el agujero XSS.

4. ¿Por qué Plantilla::escapar() rechaza objetos?

```
InvalidArgumentException: La vista recibe textos, números o arreglos; convierte los objetos
antes (DateTime).
```

Porque no hay una forma segura y general de convertir un objeto en texto: podría tener un __toString que devuelva HTML o exponer datos internos del modelo. Al rechazarlo se obliga al controlador a convertir los objetos en arreglos de textos (el método fila()), y así la vista solo recibe datos simples y no conoce el modelo.

Parte D — Errores inesperados (TODO 6)

1. Con roto.json, ¿qué ve el usuario antes del TODO 6?

```
HTTP/1.1 500 Internal Server Error
JsonException
Syntax error en /app/src/Catalogo.php:21
```

Se creó datos/roto.json con [{"tipo": "producto", "codigo": y se levantó el servidor con docker-compose.roto.yml (variable CATALOGO_ARCHIVO). El usuario veía el nombre de la clase de la excepción y el mensaje técnico con la ruta del archivo y la línea.

2. ¿Qué información útil para un atacante aparece?

La clase de la excepción (JsonException: se sabe que la aplicación lee JSON y que ese archivo está dañado), la ruta absoluta del servidor (estructura de carpetas), el nombre del archivo y la línea exacta del código (Catalogo.php:21). Con eso se puede mapear el sistema y preparar otros ataques.

3. Completa el TODO 6. ¿Qué ve ahora el usuario y qué queda en datos/errores.log?

```
HTTP/1.1 500 Internal Server Error
Algo salió mal
Tuvimos un problema inesperado. Si persiste, avisa a soporte con la referencia 3d43708a.

datos/errores.log:
[2026-10-14T23:48:33+00:00] ref=3d43708a JsonException: Syntax error en
/app/src/Catalogo.php:21
```

El usuario ve un mensaje amable y una referencia de 8 caracteres (bin2hex(random_bytes(4))), sin clase, ruta ni traza. La referencia cambia en cada error. El detalle técnico (fecha, referencia, clase, mensaje, archivo y línea) queda solo en datos/errores.log, para que el desarrollador lo investigue cuando el usuario cite la referencia.

Parte E — Verificación

php -l en los cinco archivos modificados y php verificar.php: 15 de 15 comprobaciones.

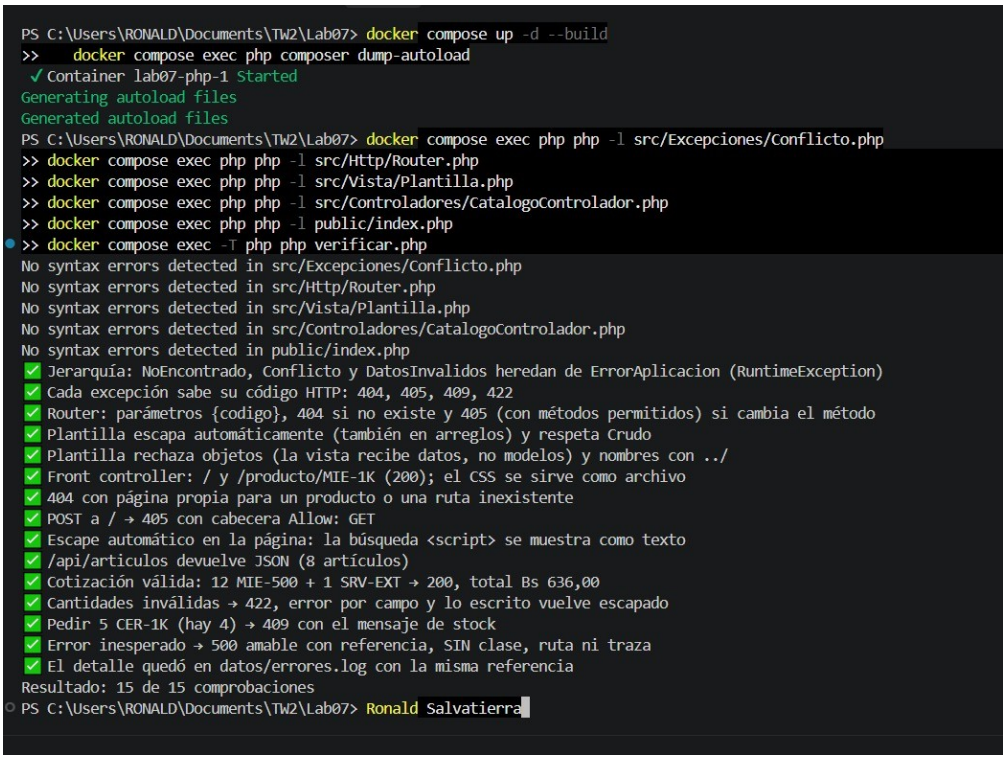


Figura 2. php -l sin errores y verificador con 15 de 15 comprobaciones.

Pruebas manuales en el navegador:



Figura 3. Catálogo en / con estilos (CSS entregado como archivo).



Figura 4. Formulario de cotización en /cotizacion.

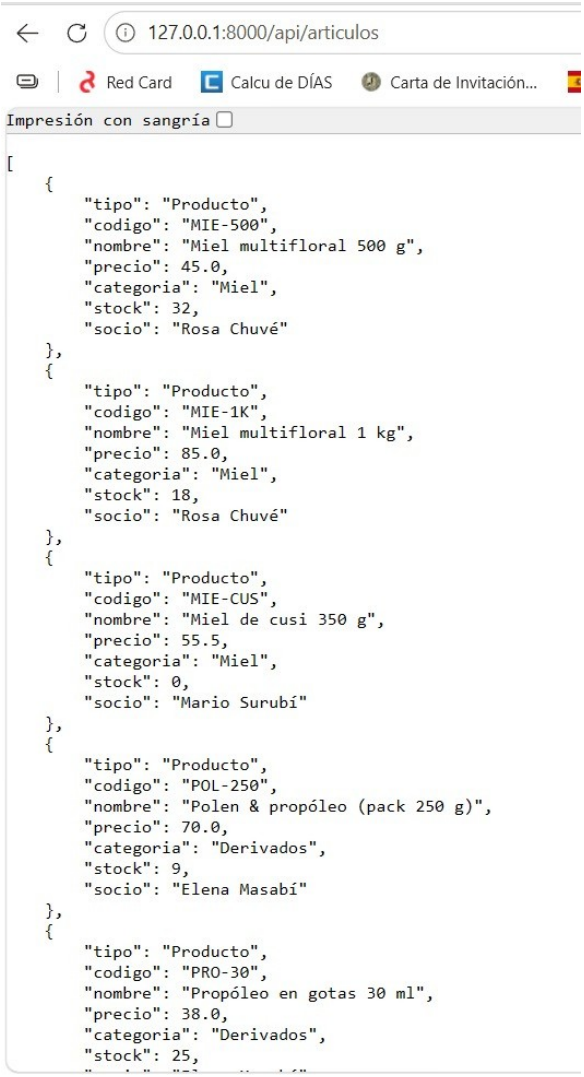


Figura 5. /api/articulos: JSON con los artículos.



Figura 6. /ruta-inexistente: página 404 propia, sin detalles internos.

Resultados de las demás pruebas manuales (ejecutadas con la misma aplicación):

```
12 × MIE-500 + 1 × SRV-EXT  →  200  12 × Miel multifloral 500 g = Bs 486,00 · 1 × Alquiler de centrífuga
= Bs 150,00 · Total Bs 636,00
5 × CER-1K (hay 4)          →  409  Stock insuficiente de CER-1K: hay 4, se pidieron 5.
cantidad -3                 →  422  "Escribe un número entero entre 0 y 1000." (el formulario conserva
lo escrito)
```

```
POST /                          →  405  Allow: GET
/?q=<script>alert(1)</script> →  200  se muestra como texto: &lt;script&gt;alert(1)&lt;/script&gt;
catálogo dañado                 →  500  "Tuvimos un problema inesperado… referencia xxxxxxxx"
```

Reflexión (preparación de la Validación 1)

Diferencia entre un error previsto (4xx) y uno inesperado (5xx), con un ejemplo, y quién debe enterarse de cada uno.

Un error previsto es una situación que la aplicación sabe que puede ocurrir y que se debe a la petición o al estado actual: pedir 5 unidades de CER-1K cuando hay 4 (409), escribir una cantidad negativa (422) o abrir una ruta que no existe (404). Su mensaje lo escribimos nosotros, es seguro mostrarlo y el usuario puede corregirlo: debe enterarse el usuario, con un mensaje claro. Un error inesperado es una falla que no anticipamos, como un catalogo.json dañado (500): el usuario no puede hacer nada y su detalle interno (clase, ruta, línea) no debe mostrarse. Debe enterarse el desarrollador, por el registro datos/errores.log, y el soporte, que recibe del usuario solo la referencia para localizar el problema.

Anexo — Archivos completados

Archivos de la carpeta inicio con los TODO resueltos. Los demás archivos (Catalogo, modelos, plantillas, verificar.php, etc.) no se modificaron.

src/Excepciones/Conflicto.php

```
<?php
declare(strict_types=1);

namespace Apicultores\Excepciones;

/** 409: la petición es válida pero choca con el estado actual (por ejemplo, no hay stock suficiente).
*/
final class Conflicto extends ErrorAplicacion
{
    // TODO 1: 409 Conflict: la petición es válida, pero choca con el estado actual del recurso (no hay
stock).
    public function codigoHttp(): int { return 409; }
    public function titulo(): string { return 'No se puede completar'; }
}
```

src/Http/Router.php

```
<?php
// src/Http/Router.php — Enrutador mínimo: relaciona "MÉTODO + ruta" con un controlador.
declare(strict_types=1);

namespace Apicultores\Http;

use Apicultores\Excepciones\MetodoNoPermitido;
use Apicultores\Excepciones\NoEncontrado;

final class Router
{
    /** @var array<int, array{metodo: string, patron: string, accion: callable}> */
    private array $rutas = [];

    public function get(string $ruta, callable $accion): void
    {
        $this->agregar('GET', $ruta, $accion);
    }

    public function post(string $ruta, callable $accion): void
    {
        $this->agregar('POST', $ruta, $accion);
    }

    /** '/producto/{codigo}' → expresión regular con un grupo con nombre: #^/producto/(?P<codigo>[^/]+)
$# */
    private function agregar(string $metodo, string $ruta, callable $accion): void
    {
        $patron = '#^' . preg_replace('#\{(\w+)\}#', '(?P<$1>[^/]+)', $ruta) . '$#';
        $this->rutas[] = ['metodo' => $metodo, 'patron' => $patron, 'accion' => $accion];
    }

    /** Ejecuta el controlador que corresponde y devuelve lo que este devuelva (HTML o datos). */
```

```
    public function despachar(string $metodo, string $uri): mixed
    {
        $ruta = rawurldecode((string) parse_url($uri, PHP_URL_PATH));
        $permitidos = [];
        foreach ($this->rutas as $r) {
            if (!preg_match($r['patron'], $ruta, $coincide)) {
                continue;
            }
            if ($r['metodo'] !== $metodo) {
                // TODO 2: la ruta EXISTE pero con otro método: se anota cuál sí acepta (irá en la
cabecera Allow)
                $permitidos[] = $r['metodo'];
                continue;
            }
            $parametros = array_filter($coincide, 'is_string', ARRAY_FILTER_USE_KEY);
            return ($r['accion'])(...$parametros);  // argumentos con nombre: $codigo
        }
        if ($permitidos !== []) {
            throw new MetodoNoPermitido(array_values(array_unique($permitidos)));
        }
        throw new NoEncontrado("No existe la página $ruta.");
    }
}
```

src/Vista/Plantilla.php

```
<?php
// src/Vista/Plantilla.php — Motor de plantillas mínimo con ESCAPE AUTOMÁTICO (la idea de Twig y Blade).
// La plantilla recibe los datos ya escapados: olvidar e() deja de ser un agujero de seguridad.
declare(strict_types=1);

namespace Apicultores\Vista;

use InvalidArgumentException;

final class Plantilla
{
    public function __construct(private readonly string $carpeta)
    {
    }

    /** Renderiza plantillas/$nombre.php dentro de plantillas/base.php. */
    public function pagina(string $nombre, array $datos = [], string $titulo = ''): string
    {
        $contenido = $this->parcial($nombre, $datos);
        return $this->parcial('base', ['titulo' => $titulo, 'contenido' => new Crudo($contenido)]);
    }

    public function parcial(string $nombre, array $datos = []): string
    {
        if (!preg_match('/^[a-z_]+$/', $nombre) || !is_file($archivo = "{$this->carpeta}/$nombre.php"))
{
            throw new InvalidArgumentException("No existe la plantilla $nombre.");
        }
        $variables = self::escapar($datos);
        return (static function (string $__archivo, array $__variables): string {
            extract($__variables, EXTR_SKIP);   // cada clave es una variable en la plantilla
```

```
            ob_start();                         // captura el HTML en lugar de enviarlo
            try {
                require $__archivo;
                return (string) ob_get_contents();
            } finally {
                ob_end_clean();                 // se limpia SIEMPRE, aunque la plantilla falle
            }
        })($archivo, $variables);
    }

    /** Escapa recursivamente los textos; los números y booleanos pasan igual; Crudo pasa sin tocar. */
    public static function escapar(mixed $valor): mixed
    {
        return match (true) {
            // TODO 3: los textos salen escapados (neutraliza <, >, &, comillas) antes de llegar a la
plantilla
            is_string($valor) => htmlspecialchars($valor, ENT_QUOTES, 'UTF-8'),
            is_array($valor) => array_map(self::escapar(...), $valor),
            $valor instanceof Crudo, is_int($valor), is_float($valor), is_bool($valor), $valor === null
=> $valor,
            default => throw new InvalidArgumentException(
                'La vista recibe textos, números o arreglos; convierte los objetos antes (' .
get_debug_type($valor) . ').'),
        };
    }
}
```

src/Controladores/CatalogoControlador.php

```
<?php
// src/Controladores/CatalogoControlador.php — Recibe la petición, usa el modelo y elige la vista. Nada
de HTML aquí.
declare(strict_types=1);

namespace Apicultores\Controladores;

use Apicultores\Catalogo;
use Apicultores\Contratos\Vendible;
use Apicultores\Excepciones\Conflicto;
use Apicultores\Excepciones\DatosInvalidos;
use Apicultores\Excepciones\NoEncontrado;
use Apicultores\Modelo\Producto;
use Apicultores\Vista\Plantilla;
use DomainException;

final class CatalogoControlador
{
    public function __construct(private readonly Catalogo $catalogo, private readonly Plantilla $vista)
    {
    }

    public function inicio(): string
    {
        $q = trim((string) ($_GET['q'] ?? ''));
        $articulos = array_filter($this->catalogo->todos(),
            fn(Vendible $a): bool => $q === '' || mb_stripos($a->nombre(), $q) !== false);
        return $this->vista->pagina('catalogo', [
```

```
            'q' => $q,
            'articulos' => array_map($this->fila(...), array_values($articulos)),
            'inventario' => bs($this->catalogo->valorInventario()),
        ], 'Catálogo');
    }

    public function producto(string $codigo): string
    {
        $a = $this->catalogo->buscar($codigo) ?? throw new NoEncontrado("No existe el producto
$codigo.");
        return $this->vista->pagina('producto', $this->fila($a), $a->nombre());
    }

    /** @return array<int, array> datos para la API (json_encode usa jsonSerialize) */
    public function api(): array
    {
        return $this->catalogo->todos();
    }

    public function formularioCotizacion(): string
    {
        return $this->mostrarCotizacion([], []);
    }

    public function cotizar(): string
    {
        $cantidades = (array) ($_POST['cantidad'] ?? []);
        $pedido = [];
        $errores = [];
        foreach ($cantidades as $codigo => $texto) {
            $codigo = (string) $codigo;
            if ($this->catalogo->buscar($codigo) === null) {
                continue;                                       // se ignoran códigos que no existen
            }
            $n = filter_var(trim((string) $texto) === '' ? '0' : $texto, FILTER_VALIDATE_INT, ['options'
=> ['min_range' => 0, 'max_range' => 1000]]);
            if ($n === false) {
                $errores[$codigo] = 'Escribe un número entero entre 0 y 1000.';
            } elseif ($n > 0) {
                $pedido[$codigo] = $n;
            }
        }
        if ($errores === [] && $pedido === []) {
            $errores['general'] = 'Indica al menos una cantidad.';
        }
        if ($errores !== []) {
            throw new DatosInvalidos($errores);
        }
        // TODO 5: el modelo lanza DomainException (regla de negocio); el controlador la traduce a
Conflicto (HTTP 409)
        //         y conserva la original como $previous para no perder la causa.
        try {
            $this->catalogo->verificarStock($pedido);
        } catch (DomainException $e) {
            throw new Conflicto($e->getMessage(), 0, $e);
        }
```

```
        return $this->mostrarCotizacion($cantidades, [], $pedido);
    }

    /** Se usa también desde el manejador de errores para volver a mostrar el formulario con los errores
(422). */
    public function mostrarCotizacion(array $cantidades, array $errores, array $pedido = []): string
    {
        $lineas = [];
        foreach ($pedido as $codigo => $n) {
            $a = $this->catalogo->buscar($codigo);
            $lineas[] = ['nombre' => $a->nombre(), 'cantidad' => $n, 'subtotal' => bs($a-
>precioVenta($n))];
        }
        return $this->vista->pagina('cotizacion', [
            'articulos' => array_map($this->fila(...), $this->catalogo->disponibles()),
            'cantidades' => array_map('strval', $cantidades),
            'errores' => $errores,
            'lineas' => $lineas,
            'total' => $pedido === [] ? null : bs($this->catalogo->totalPedido($pedido)),
        ], 'Cotización');
    }

    /** Convierte un objeto en un arreglo de textos para la vista (la vista no recibe objetos). */
    private function fila(Vendible $a): array
    {
        return [
            'codigo' => $a->codigo(), 'nombre' => $a->nombre(), 'tipo' => $a->tipo(),
            'precio' => $a->precioFormateado(), 'detalle' => $a->detalle(),
            'categoria' => $a instanceof Producto ? $a->categoria()->value : 'Servicios',
        ];
    }
}
```

public/index.php

```
<?php
// public/index.php — FRONT CONTROLLER: la ÚNICA puerta de entrada. Toda petición pasa por aquí.
//   Servidor de desarrollo:  php -S 127.0.0.1:8000 -t public public/index.php
//   Nginx (Tecnología Web I): location / { try_files $uri /index.php$is_args$args; }
declare(strict_types=1);

// Con php -S: si piden un archivo que existe (CSS, imágenes), que lo entregue el servidor
// TODO 4: con 'cli-server', si la ruta pedida es un archivo real dentro de public/ (CSS, imágenes),
//         se devuelve false y el servidor lo entrega tal cual. realpath() evita salirse de public/ con
"../".
if (PHP_SAPI === 'cli-server') {
    $archivo = realpath(__DIR__ . rawurldecode((string) parse_url($_SERVER['REQUEST_URI'],
PHP_URL_PATH)));
    if ($archivo !== false && str_starts_with($archivo, realpath(__DIR__) . DIRECTORY_SEPARATOR) &&
is_file($archivo)) {
        return false;
    }
}

require __DIR__ . '/../vendor/autoload.php';

use Apicultores\Catalogo;
```

```
use Apicultores\Controladores\CatalogoControlador;
use Apicultores\Excepciones\DatosInvalidos;
use Apicultores\Excepciones\ErrorAplicacion;
use Apicultores\Excepciones\MetodoNoPermitido;
use Apicultores\Http\Router;
use Apicultores\Vista\Plantilla;

ini_set('display_errors', '0');                  // en producción NUNCA se muestran detalles internos
const REGISTRO_ERRORES = __DIR__ . '/../datos/errores.log';

// Avisos y advertencias de PHP también se convierten en excepciones: nada falla "a medias"
set_error_handler(function (int $nivel, string $mensaje, string $archivo, int $linea): bool {
    throw new ErrorException($mensaje, 0, $nivel, $archivo, $linea);
});

$vista = new Plantilla(__DIR__ . '/../plantillas');

try {
    $catalogo = Catalogo::desdeArchivo(getenv('CATALOGO_ARCHIVO') ?: __DIR__ .
'/../datos/catalogo.json');
    $c = new CatalogoControlador($catalogo, $vista);

    $router = new Router();
    $router->get('/', $c->inicio(...));
    $router->get('/producto/{codigo}', $c->producto(...));
    $router->get('/api/articulos', $c->api(...));
    $router->get('/cotizacion', $c->formularioCotizacion(...));
    $router->post('/cotizacion', $c->cotizar(...));

    $resultado = $router->despachar($_SERVER['REQUEST_METHOD'], $_SERVER['REQUEST_URI']);
    if (is_array($resultado)) {
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode($resultado, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE |
JSON_PRESERVE_ZERO_FRACTION | JSON_THROW_ON_ERROR);
    } else {
        echo $resultado;
    }
} catch (DatosInvalidos $e) {
    // 422: se vuelve a mostrar el formulario con lo que escribió el usuario y los errores por campo
    http_response_code($e->codigoHttp());
    echo $c->mostrarCotizacion((array) ($_POST['cantidad'] ?? []), $e->errores());
} catch (ErrorAplicacion $e) {
    // Errores PREVISTOS (404, 405, 409): su mensaje lo escribimos nosotros, es seguro mostrarlo
    http_response_code($e->codigoHttp());
    if ($e instanceof MetodoNoPermitido) {
        header('Allow: ' . implode(', ', $e->permitidos()));
    }
    echo $vista->pagina('error', ['codigo' => $e->codigoHttp(), 'titulo' => $e->titulo(), 'mensaje' =>
$e->getMessage()], $e->titulo());
} catch (Throwable $e) {
    // Errores INESPERADOS (500): el detalle va al registro; al usuario, solo un número de referencia
    // TODO 6: el detalle técnico (clase, mensaje, archivo, línea) va SOLO al registro; el usuario
recibe una referencia.
    $referencia = bin2hex(random_bytes(4));
    error_log(sprintf("[%s] ref=%s %s: %s en %s:%d\n", date('c'), $referencia, $e::class,
        $e->getMessage(), $e->getFile(), $e->getLine()), 3, REGISTRO_ERRORES);
```

```
    http_response_code(500);
    echo $vista->pagina('error', ['codigo' => 500, 'titulo' => 'Algo salió mal',
        'mensaje' => "Tuvimos un problema inesperado. Si persiste, avisa a soporte con la referencia
$referencia."], 'Error');
}
```